

Lab : Load Balancer Installation And Traffic Simulation (Linux)

Nephtalie Twite Wa Mwamba
202402067
Software Engineering

1 Installation Proof

```
> haproxy -v
HAProxy version 3.2.14-1 2026/03/09 - https://haproxy.org/
Status: long-term supported branch - will stop receiving fixes around Q2 2030.
Known bugs: http://www.haproxy.org/bugs/bugs-3.2.14.html
Running on: Linux 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1
(2026-02-25) x86_64
```

2. Server Startup Evidence

server 1

```
> echo "Server 1 - 202402067" > index.html
> python3 -m http.server 8023
Serving HTTP on 0.0.0.0 port 8023 (http://0.0.0.0:8023/) ...
127.0.0.1 - - [26/Mar/2026 13:20:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 13:27:34] "GET / HTTP/1.1" 200 -
```

server 2

```
echo "Server 2 - 202402067" > index.html
python3 -m http.server 8024
Serving HTTP on 0.0.0.0 port 8024 (http://0.0.0.0:8024/) ...
127.0.0.1 - - [26/Mar/2026 13:20:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 13:27:34] "GET / HTTP/1.1" 200 -
```

server 3

```
echo "Server 3 - 202402067" > index.html
python3 -m http.server 8025
Serving HTTP on 0.0.0.0 port 8025 (http://0.0.0.0:8025/) ...
127.0.0.1 - - [26/Mar/2026 13:20:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 13:27:34] "GET / HTTP/1.1" 200 -
```

3 Unique Server Identify Proof

```
> cat index.html
Server 1 - 202402067
```

```
> cat index.html
Server 2 - 202402067
```

```
> cat index.html
Server 3 - 202402067
```

4. HAProxy configuration Proof

```
cat /etc/haproxy/haproxy.cfg | tail -n 15
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http
```

```
frontend http_front
  bind *:8123
  default_backend servers
```

```
backend servers
    balance roundrobin
    server s1 127.0.0.1:8023 check
    server s2 127.0.0.1:8024 check
    server s3 127.0.0.1:8025 check
```

5 Load Distribution Evidence

```
for i in {1..10}; do curl http://localhost:8024; done
```

[illegible]

6. Load Testing Output

```
> ab -n 100 -c 10 http://127.0.0.1:8123/
```

This is ApacheBench, Version 2.3 <\$Revision: 1923142 \$>

Copyright 1996 Adam Twiss, Zeus Technology Ltd, <http://www.zeustech.net/>

Licensed to The Apache Software Foundation, <http://www.apache.org/>

Benchmarking 127.0.0.1 (be patient).....done

Server Software: SimpleHTTP/0.6

Server Hostname: 127.0.0.1

Server Port: 8123

Document Path: /

Document Length: 21 bytes

Concurrency Level: 10

Time taken for tests: 0.043 seconds

Complete requests: 100

Failed requests: 0

Total transferred: 20700 bytes

HTML transferred: 2100 bytes

Requests per second: 2309.63 [#/sec] (mean)

Time per request: 4.330 [ms] (mean)

Time per request: 0.433 [ms] (mean, across all concurrent requests)

Transfer rate: 466.89 [Kbytes/sec] received

Connection Times (ms)

	min	mean	mean[+/-sd]	median	max
Connect:	0	0	0.1	0	1
Processing:	1	4	1.6	4	10
Waiting:	1	4	1.5	4	10
Total:	1	4	1.6	4	10

Percentage of the requests served within a certain time (ms)

50%	4
66%	5
75%	5
80%	5
90%	6
95%	7
98%	8
99%	10
100%	10 (longest request)

7. Failure Test Evidence

```
> for i in {1..10}; do curl http://localhost:8123; done
Server 3 - 202402067
Server 1 - 202402067
Server 3 - 202402067
Server 1 - 202402067
Server 3 - 202402067
Server 1 - 202402067
Server 3 - 202402067
Server 1 - 202402067
Server 3 - 202402067
Server 1 - 202402067
```

8. Process Verification

```
> ps aux | grep haproxy
root    10142  0.0  0.4 63156 31948 ?        Ss   13:25   0:00 /usr/sbin/haproxy -Ws -f
/etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy-master.sock
haproxy  10144  0.0  1.0 214140 85096 ?        Sl   13:25   0:00 /usr/sbin/haproxy -Ws -f
/etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy-master.sock
happy    15543  0.0  0.0  6548  2496 pts/3    S+   14:03   0:00 grep --color=auto
--exclude-dir=.bzip --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn
--exclude-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv haproxy
```

Reflection

1. HAProxy used the Round Robin algorithm.
2. Requests are distributed to balance the load, improve performance, and ensure no single server is overloaded.
3. After stopping one server, HAProxy stopped sending traffic to it and redirected requests to the remaining active servers, ensuring continuous availability.